НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): студентка групи АІ-241
Шевель Ольга Олександрівна

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	209	5
Час, с	28,33	9,09
Токенів/с	11,94	25,50

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1244	176
Час, с	114,69	8,53

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: Токени міркувальної моделі пішли на внутрішній аналіз задачі, перевірку можливих варіантів та формування відповіді. Для простого фактичного питання такі додаткові міркування не дають помітної переваги.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі мають перевагу в складних математичних, логічних задачах, програмуванні та задачах, де потрібно виконати кілька кроків

міркування. Для простих фактів, перекладу або коротких відповідей їх використання часто невиправдане через більший час і витрати токенів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі переважно навчаються виконувати команди та давати корисні відповіді на запити користувача. Міркувальні моделі додатково навчаються ефективно виконувати багатокрокове міркування та знаходити правильне рішення складних задач.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість токенів не означає автоматично, що модель розумніша. Важливішими показниками є правильність відповіді, здатність розв’язувати складні задачі та ефективність міркування.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Llama 3.2:3B генерує токени швидше, зокрема через менший розмір моделі та відсутність тривалого міркування. Qwen3:4B витрачає більше обчислювальних ресурсів на reasoning, тому її швидкість генерації нижча.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 41,8 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 917,5 с проти 68,2 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Один і той самий запит виконано для моделей llama3.2:3b та qwen3:4b з двома варіантами system-повідомлення: «Ти лаконічний технічний асистент.» і «Відповідай лише українською, максимум двома реченнями.». Кількість токенів взято з response.usage.

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	284	українська	2	так
llama3.2:3b	українською, 2 речення	291	українська	2	так
qwen3:4b	лаконічний асистент	188	англійська	2	так
qwen3:4b	українською, 2 речення	254	українська	2	так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	57	86	143
llama3.2:3b	українською, 2 речення	61	73	134
qwen3:4b	лаконічний асистент	61	2196	2257
qwen3:4b	українською, 2 речення	70	987	1057

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Точніше обмеження «максимум двома реченнями» дотримались обидві моделі: у всіх випадках було рівно 2 речення.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Так, qwen3:4b з лаконічним system-повідомленням відповіла англійською, хоча очікувалась українська. Після уточнення system-повідомлення модель вже відповіла українською.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: prompt_tokens збільшився після додавання вимог про українську мову та 2 речення, оскільки system-повідомлення стало довшим. Наприклад, у llama3.2:3b значення зросло з 57 до 61, а в qwen3:4b з 61 до 70.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: У qwen3:4b completion_tokens значно більший через особливості токенізації та внутрішньої генерації моделі, зокрема можливе врахування прихованих міркувань. Тому кількість токенів генерації може бути набагато більшою за кількість токенів у видимій відповіді.

Відповіді на питання чат-бота (крок 7)

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Інструктивна та міркувальна моделі можуть по-різному виконувати однакові інструкції через відмінності в архітектурі, навчанні та способі генерації відповіді. У наведеному прикладі qwen3:4b без чіткої вказівки в system-повідомленні відповіла англійською, тоді як llama3.2:3b дотрималась української мови.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення задає основні правила та контекст роботи моделі, а user-повідомлення містить безпосередній запит користувача. Зазвичай system має вищий пріоритет, тому його інструкції мають виконуватися перед інструкціями user, якщо вони суперечать одна одній.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Міркувальна модель може генерувати додаткові внутрішні токени під час побудови відповіді, які не відображаються користувачу. Тому в qwen3:4b кількість completion_tokens може бути значно більшою, навіть якщо видима відповідь має приблизно таку саму довжину.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Якщо інструкції system і user суперечать одна одній, модель зазвичай надає перевагу system-повідомленню. Наприклад, якщо system вимагає відповідати українською, а user просить відповідати англійською, модель повинна дотримуватися системної інструкції.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі мовні моделі мають менше параметрів і зазвичай слабше навчені на різноманітних інструкціях та мовних контекстах. Через це вони можуть частіше ігнорувати окремі вимоги, наприклад мову або формат відповіді.

Висновок за завданням 2

System-повідомлення є важливим способом керування поведінкою моделі, але воно не гарантує стовідсоткового виконання інструкцій. Самої інструкції іноді недостатньо, тому варто формулювати її максимально конкретно, перевіряти результат і за потреби повторювати або уточнювати вимоги.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Один і той самий запит виконано через єдиний OpenAI-сумісний інтерфейс для двох провайдерів: локальної моделі llama3.2:3b (Ollama) та хмарної gemini-3.5-flash. Усі відмінності між провайдерами зібрано у словнику PROVIDERS у файлі providers.py.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash
Час відповіді, с	11,38	13,50
completion_tokens	173	351
Довжина відповіді, символів	617	1037
Мова відповіді	українська	українська
Фактичних помилок у відповіді	8	3

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: У compare.py змін немає, бо create() викликається однаково. Змінилися лише три значення в словнику PROVIDERS (base_url, api_key, model), а ключ береться з .env.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна повніша й точніша: 1037 символів проти 617, і 3 фактичні помилки проти 8.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, у відповіді llama3.2:3b було 8 фактичних помилок. Українську вона загалом тримала, але в Завданні 5 у неї були русизми й спотворена граматика.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого?

Відповідь: За твоїми даними загальний час хмарної був більший (13,50 с проти 11,38 с), тож «швидше» тут неточне. Але вона згенерувала вдвічі більше токенів (351 проти 173), тобто ≈26 ток/с проти ≈15 ток/с. Швидкість генерації вища завдяки потужним серверним GPU, і це майже компенсує мережеву затримку.

Відповіді на питання чат-бота (крок 6)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI API має простий та зручний формат для роботи з мовними моделями. Його підтримують інші провайдери, щоб розробники могли використовувати вже написаний код без значних змін.

2. Я порівнювала локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Кількість параметрів може впливати на здатність моделі розуміти складні запити та формувати якісніші відповіді. Але на результат також впливають архітектура моделі, якість навчальних даних, налаштування та тип конкретного завдання.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарні моделі працюють на потужних серверах зі спеціалізованим обладнанням для обробки нейромереж. Тому висока швидкість обчислень на сервері може компенсувати час передачі даних через мережу.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Під час використання хмарного API дані запиту передаються на зовнішній сервер, тому існує ризик розкриття конфіденційної інформації. У комерційних проєктах використовують шифрування, контроль доступу та обмежують передачу персональних і конфіденційних даних.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit – це обмеження на кількість запитів або використання API за певний проміжок часу. Розробники можуть зменшувати кількість запитів, використовувати кешування, черги запитів або перейти на тариф із більшими лімітами.

Висновок за завданням 3

Локальну модель я б обрала для роботи з конфіденційними даними та коли потрібна робота без доступу до Інтернету. Хмарну модель я б обрала для складних завдань, де важливі якість відповіді та більші можливості моделі.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Швидкість виміряно у стрімінговому режимі (stream=True) для трьох промптів: короткого, середнього та довгого. Перед серією вимірів виконано прогрівальний запит, кожен вимір повторено тричі, у таблиці наведено медіану.

Таблиця 7 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2,097	2,690	8,9
локальна	llama3.2:3b	середній	2,230	7,086	42,3
локальна	llama3.2:3b	довгий	2,269	29,020	64,4
хмарна	gemini-3.5-flash	короткий	8,777	8,816	3,2
хмарна	gemini-3.5-flash	середній	14,700	15,511	36,4
хмарна	gemini-3.5-flash	довгий	18,567	24,502	143,3

Таблиця 8 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2,277	2,165
Всього, с	2,889	2,862

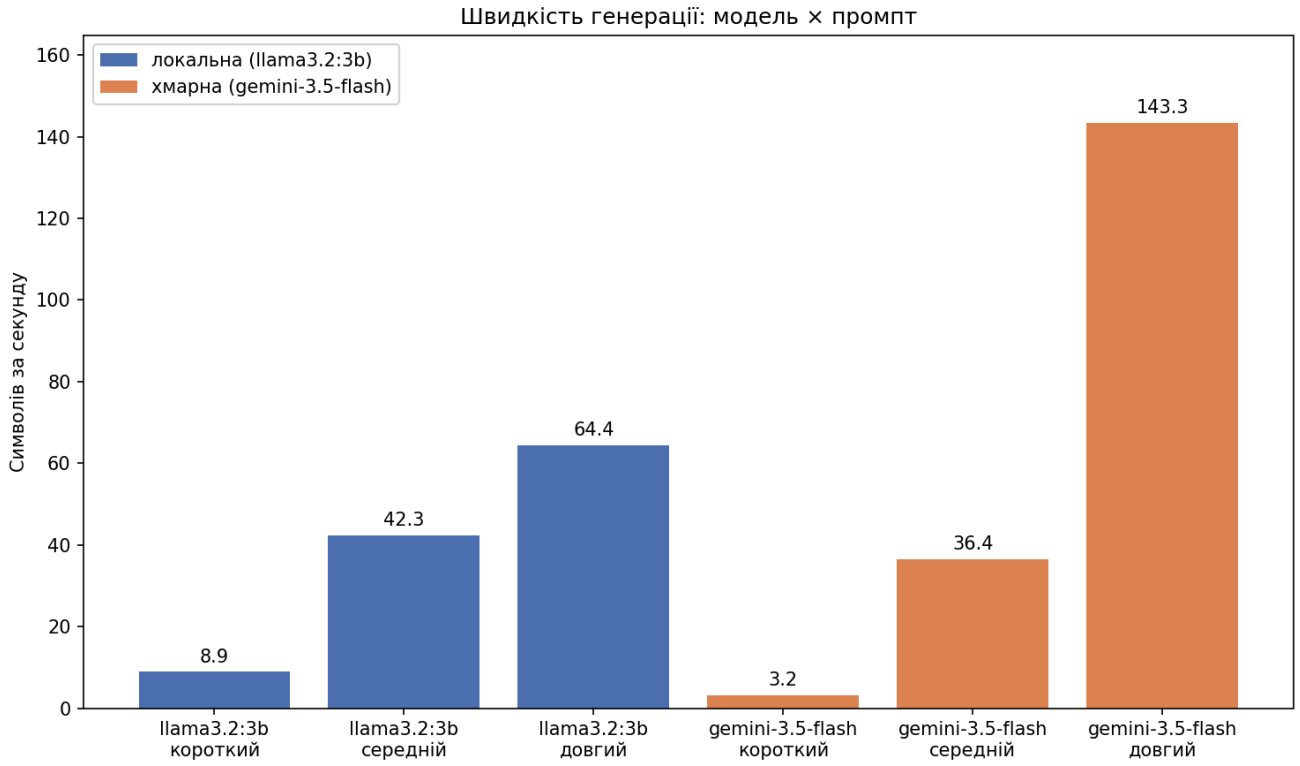


Рисунок 1 – Швидкість генерації (символів за секунду): модель × промпт

Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий (значно нижчий) TTFT показала локальна модель, оскільки її запити обробляються безпосередньо на пристрої без затримок мережевої передачі даних, черг на сервері та накладних витрат хмарного API.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Ні, не збігається. Локальна модель перемагає за TTFT завдяки миттєвому початку обробки, але хмарна модель суттєво випереджає її за

швидкістю генерації (символів на секунду) завдяки використанню потужніших серверних графічних прискорювачів.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: На коротких промптах локальна модель виявилася швидшою за загальним часом виконання через відсутність затримки мережі. Проте на довгих промптах хмарна модель випереджає локальну за загальним часом (24,5 с проти 29,0 с) завдяки значно вищій швидкості генерації тексту (143,3 проти 64,4 симв./с).

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Повтори можуть мати незначні відхилення через нестабільність мережевого з'єднання або коливання завантаження процесора/пам'яті системою. Одного запуску недостатньо, а усереднення трьох повторів за допомогою медіани дозволяє відсіяти випадкові сплески затримок.

5. Чому холодний старт повільніший за теплий?

Відповідь: При холодному старті витрачається додатковий час на ініціалізацію середовища та зчитування вагових коефіцієнтів моделі з накопичувача в оперативну або відеопам'ять. При теплому старті модель уже знаходиться в пам'яті й готова до обробки запиту без затримок.

Відповіді на питання чат-бота (крок 7)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT відбувається фаза обробки вхідного промпту (prefill): модель токенізує текст, будує кеш ключів та значень (KV cache), обчислює перший токен та передає його через мережу. Цей показник вимірюють окремо, оскільки він визначає інтерактивність і сприйняття чутливості системи користувачем, тоді як швидкість генерації показує її пропускну здатність під час виводу.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Хмарна модель має вищий TTFT через мережеву затримку (RTT), черги на сервері та накладні витрати API на ініціалізацію сесії. Проте під час самої генерації потужні серверні графічні прискорювачі (GPU/TPU) обробляють кожен наступний токен значно швидше за локальне обладнання, забезпечуючи високу швидкість у символах за секунду.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт – це стан, коли вагові коефіцієнти моделі ще не завантажені в оперативну або відеопам’ять (VRAM). Перший запуск повільніший, оскільки системі потрібно зчитати гігабайти даних із диска та ініціалізувати графічний драйвер, тоді як при наступних «теплих» запусках модель уже перебуває в пам’яті й готова до роботи.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Окремі виміри можуть зазнавати впливу мережевих коливань, фонових процесів операційної системи або тимчасових сплесків завантаження процесора. Медіана дозволяє отримати об’єктивну оцінку продуктивності, оскільки вона стійка до випадкових аномальних викидів і затримок, на відміну від середнього арифметичного.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Обробка вхідного промпту виконується паралельно для всіх токенів одночасно за рахунок ефективних матричних обчислень на GPU. Генерація ж відповіді є послідовним (авторегресійним) процесом, де для створення кожного нового токена потрібен окремий повний прохід через усю нейромережу, що створює обмеження за пропускною здатністю пам’яті.

Висновок за завданням 4

Для інтерактивного чат-бота важливішим є TTFT (час до першого токена). Користувачеві психологічно важливо швидко побачити початок відповіді – це створює відчуття миттєвого відгуку та живого діалогу, навіть якщо подальший текст друкується з помірною швидкістю.

Для фонової обробки великих обсягів тексту важливіша швидкість генерації (симв./с) та загальний час. У пакетній обробці (наприклад, сумаризація сотень документів) початкова затримка в кілька секунд не має значення, а вирішальну роль відіграє висока пропускна здатність моделі для якнайшвидшого виконання всього обсягу завдань.

Завдання 5. Оцінювання якості моделей за рубрикою

Якість оцінено за рубрикою з шести завдань і трибальною шкалою (0–2 бали) для трьох моделей: llama3.2:3b, qwen3:4b та хмарної gemini-3.5-flash. Кожне завдання виконано двічі (quality.py), відповіді збережено у quality.json.

Таблиця 13 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	2	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	1	2	1
5. Логіка	1	2	2
6. Українська	1	2	2
Разом /12	5	12	11

Таблиця 14 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	так	ні	ні
5. Логіка	так	ні	ні
6. Українська	ні	ні	ні

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрала модель qwen3:4b (12/12), випередивши Gemini (11/12) та Llama 3.2 (5/12). Основна різниця виникла у завданнях на фактичну точність (завдання 1), стиснення зі збереженням усіх чисел (завдання 3) та суворе дотримання формату «лише код» без markdown (завдання 4).

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Оцінки розійшлися лише у моделі llama3.2:3b у завданнях 4 (Код) та 5 (Логіка). Це свідчить про високу варіативність та низьку надійність малих моделей при розв’язанні логічних та синтаксичних задач.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три моделі успішно виконали цю вимогу у завданні 2, видавши чистий JSON-рядок без markdown-обгортки ```json.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Суттєві проблеми виявилися у llama3.2:3b (численні русизми на кшталт «в клітках», «недостаточна», «окончательна» та спотворена граматика). У qwen3:4b зустрічалися лише поодинокі русизми («ожирення»), тоді як Gemini 3.5 Flash продемонструвала бездоганну українську мову.

Приклад провалу (крок 7)

Запит: «У лікарні 40 пацієнтів, 18 з температурою, 15 з кашлем, 7 з обома симптомами. У скількох пацієнтів немає симптомів? Покажи хід розв'язання.»

Відповідь моделі llama3.2:3b: «1. Визначте загальну кількост пацієнтів у лікарні: 40 2. Визначте кількість пацієнтів з температурою: 18 3. Визначте кількість пацієнтів з кашлем: 15 4. Визначте кількість пацієнтів, які мають оба симптоми: 7 5. Побачите, що кількості пацієнтів, які мають лише одне симптом, не зазначено. Таким образом, їх-number не можем дизальновати. Або кількост пацієнтів без жодного симптомів.»

Правильна відповідь: 14 пацієнтів: $40 - (18 + 15 - 7) = 14$.

Пояснення: Провал стався через те, що модель 3B не змогла застосувати принцип включень-виключень із теорії множин і помилково вирішила, що даних у тексті недостатньо. Через логічний тупик у моделі деградувала якість мовного декодера, що спричинило суржик («оба», «таким образом»), вигадування слів-галюцинацій («дизальновати») та недоречні англіцизми («їх-number»).

Відповіді на питання чат-бота (крок 8)

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: На практиці використовують автоматизовані бенчмарки зі стандартизованими датасетами (MMLU, GSM8K, HumanEval) для перевірки конкретних дисциплін та навичок програмування. Також поширений підхід LLM-as-a-Judge, де потужніша модель (наприклад, GPT-4) оцінює відповіді інших моделей за заданими критеріями, а також сліпе людське оцінювання (Human Evaluation) на публічних платформах типу LMSYS Chatbot Arena. Для автоматичної порівняльної оцінки тексту застосовують семантичні та сумаризаційні метрики, такі як BERTScore, ROUGE та BLEU.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Під час базового навчання та етапу Instruction Tuning моделі аналізують мільйони прикладів, де кодові блоки та JSON завжди обгортаються у синтаксис ```json. Оскільки модель працює за принципом передбачення наступного токена, стійка математична асоціація між структурованою дужкою { і markdown-тегами переважає текстову інструкцію в промпті. Повністю вирішити цю проблему дозволяє лише примусове обмеження схеми виводу на рівні API (Structured Outputs або JSON mode).

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Основна причина – суттєвий дисбаланс у навчальних датасетах, де англійська мова займає понад 80–90% обсягу, а українська належить до мов із низьким обсягом ресурсів (low-resource languages). Через це токенізатори малих моделей розбивають українські слова на багато дрібних суб-токенів або окремих байтів, що виснажує контекстне вікно та знижує якість граматичного декодування. У результаті малим моделям бракує внутрішніх семантичних зв’язків для побудови стилістично чистої мови без суржику та русизмів.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Це зумовлено ймовірнісною природою генерації та параметром temperature (температура), який додає випадковості під час вибору кожного наступного токена. Навіть незначне відхилення у виборі першого слова змінює подальший вектор роздумів і контекст моделі. Якщо температура вища за 0, модель обирає з розподілу ймовірностей різні токени, що призводить до відмінностей у логіці, структурі чи синтаксисі між прогонами.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація – це ситуація, коли модель впевнено генерує фактологічно хибний або вигаданий контент, який зовні виглядає правдоподібно. Архітектура Transformer оптимізована для створення переконливого та зв’язного

тексту, а не для перевірки об’єктивної істини чи звірки з базою даних. Оскільки під час навчання генерація будь-якої відповіді заохочується більше, ніж відмова, модель заповнює прогалини в знаннях статистично найбільш ймовірними словосполученнями.

Висновок за завданням 5

Оцінка за рубрикою виявилась надійною там, де перевірка об’єктивна: формат (json.loads), збереження чисел, запуск коду та кінцева відповідь у логічній задачі. Найменш формалізованою залишається оцінка якості мови, де багато залежить від суб’єктивного погляду. Два прогони показали, що нестабільною є лише llama3.2:3b (розбіжності у завданнях 4 і 5), тому для малих моделей одного запуску недостатньо. Найпоказовішими виявились завдання 1, 3 і 5: llama3.2:3b отримала на них 0, 0 і 1 бал проти 2 балів у qwen3:4b та хмарної моделі. Завдання 2 показало, що всі три моделі тримають формат JSON, тож там різниці немає. Для роботи з українськомовним текстом я б обрала Gemini 3.5 Flash, оскільки її українська була найчистішою, а з локальних моделей – qwen3:4b (12/12), але з обов’язковою перевіркою результату.

Завдання 6. Керування випадковістю: temperature і top_p

Досліджено вплив параметрів temperature і top_p на різноманітність відповідей локальної моделі llama3.2:3b (файл sampling.py). Творче завдання та фактичне питання виконувались по 5 разів для кожного значення параметра.

Таблиця 16 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0,0	1	0
temperature	0,7	5	—
temperature	1,5	5	1

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
top_p	0,1	1	—
top_p	0,5	4	—
top_p	1,0	5	—

Фактичне питання: рік відкриття пеніциліну Флемінгом. Правильна відповідь – 1928 (джерело: https://nlm.nih.gov/exhibition/fromdnatobeer/digitalgallery/detail-B07927.html).
Модель при temperature=0,0 п'ять разів відповіла «1941» (хибно), при temperature=1,5 – чотири рази «1941» і один раз «1928» (правильно).

Таблиця 17 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0,0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Для агента, який повертає JSON, я поставлю temperature 0 (щонайбільше 0,2), оскільки потрібна передбачуваність, а не різноманітність, а випадковість збільшує ризик зламаного формату.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генератора заголовків підійде temperature 0,8–1,0: потрібна різноманітність, інакше модель щоразу видаватиме ту саму нудну відповідь.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Temperature змінює контрастність усього розподілу: при високих значеннях різниця між імовірним і малоймовірним зменшується. Top_p не змінює співвідношення ймовірностей, а лише відсікає «хвіст» малоймовірних кандидатів, тому він обмежує коло вибору, а не «розм'якшує» його.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Температура 0 не гарантує абсолютної відтворюваності через особливості обчислень із плаваючою комою та паралелізацію. В експерименті з 10 запусків усі відповіді збіглися («1941»), тобто унікальних відповідей була 1 з 10, але це лише збіг у конкретному прогоні, а не гарантія. Також відтворюваність не означає правильність: модель стабільно видала хибний рік замість 1928.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Найбезглуздішою виявилась відповідь «Лічем: ЭлектронОчити!» – суміш українських, російських і випадкових слів, яка не є назвою платформи й не пов'язана із завданням. Ще одним прикладом є «Охотно, ось назва platformsky-uchenie» (російське слово й латинська транслітерація). При високій температурі розподіл ймовірностей сплющується, тому шанс отримують малоймовірні токени з інших мов і випадкові фрагменти слів. Після такого вибору наступні токени генеруються вже з урахуванням цього «збою», і помилка накопичується. Мала модель із невеликим обсягом українських даних у навчанні особливо схильна до цього.

Відповіді на питання чат-бота (крок 8)

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Перед застосуванням softmax логіти токенів ділять на значення temperature. При $T < 1$ розподіл стає різкішим і домінує найімовірніший токен, при

T > 1 він згладжується, і зростає шанс малоймовірних варіантів; при T → 0 вибір зводиться до жадібного (завжди найімовірніший токен).

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Top_p (nucleus sampling) сортує токени за ймовірністю, залишає найменшу множину, сумарна ймовірність якої досягає p, а решту відкидає. Temperature змінює форму всього розподілу, тоді як top_p обрізає його хвіст.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Зазвичай краще змінювати щось одне: обидва параметри керують випадковістю, їх ефекти накладаються, і важко зрозуміти, що саме дало результат. Типова практика – залишити top_p за замовчуванням і налаштовувати temperature.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Причина в обчисленнях із плаваючою комою та паралелізації: порядок операцій може відрізнятись, і мінімальні розбіжності в логітах здатні змінити вибір між майже рівноймовірними токенами. Далі така розбіжність накопичується, і відповіді розходяться.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних зазвичай використовують 0–0,2, щоб результат був стабільним. Для творчого тексту – приблизно 0,7–1,0, а іноді й вище, коли потрібна більша різноманітність.

Висновок за завданням 6

Для наступних лабораторних, де модель має повертати структуровані дані, я оберу низьку температуру (0–0,2) і стандартне значення top_p, оскільки потрібна передбачуваність та стабільний формат. Налаштування «за замовчуванням» (зазвичай temperature близько 0,7–1,0) можуть виявитися непридатними, адже

додають випадковість, через яку структура відповіді може порушитися, а результати двох запусків – відрізнятися.

Крок 8. Звести все разом і зробити висновок

Підсумкове порівняння трьох моделей за даними всіх попередніх завдань зведено в таблицю 18. Значення швидкості та TTFT узято із завдань 1 і 4, бали – із завдання 5.

Таблиця 18 – Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3b)	Локальна більша (qwen3:4b)	Хмарна (gemini-3.5-flash)
Швидкість генерації	25,5 ток/с; 64,4 симв./с (довгий промпт)	11,94 ток/с (витрачає токени на міркування)	143,3 симв./с (довгий промпт)
TTFT	≈2,1–2,3 с	не вимірювався	8,8–18,6 с
Якість, балів /12	5	12	11
Стабільність формату (завдання 2)	JSON без markdown, обидва прогони	так само	так само
Потрібні ресурси	≈2 ГБ пам’яті, працює на ЦП	≈2,5 ГБ пам’яті, багато токенів на міркування	локально майже нічого; потрібні мережа й API-ключ
Вартість	безкоштовно (електроенергія)	безкоштовно (електроенергія)	безкоштовний тариф з rate limit, далі оплата за токени
Куди йдуть ваші дані	залишаються на пристрої	залишаються на пристрої	на сервер провайдера
Працює без інтернету	так	так	ні
Коли обирати саме її	прості завдання, конфіденційні дані, потрібен швидкий перший токен (інтерактивний чат)	конфіденційні дані та потрібні якість і міркування, коли час відповіді не критичний	складні завдання, довгі тексти, найчистіша українська; дані не конфіденційні

Правило вибору моделі

Локальну модель я обираю, коли дані конфіденційні, потрібна робота без інтернету, потрібен швидкий перший токен (інтерактивний чат) або завдання просте й добре формалізоване. Слабкість цього варіанта – якість: llama3.2:3b набрала лише 5 балів із 12, тому її результат потрібно перевіряти, а для складніших завдань краще брати локальну qwen3:4b (12 балів), якщо час відповіді не критичний.

Хмарну модель я обираю, коли важливі якість і чиста українська мова, текст довгий і потрібна висока швидкість генерації, а дані не конфіденційні. Ціна такого вибору – гірший TTFT (9–19 с проти ≈2 с), залежність від мережі, ліміти безкоштовного тарифу та передача даних назовні.

Порядок прийняття рішення: спочатку питання про дані (чи можна їх відправляти назовні), потім про складність завдання, потім про чутливість до затримки. Незалежно від моделі формат відповіді слід перевіряти програмно, а temperature для структурованих даних тримати низькою (0–0,2).

Загальні висновки

Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій системи потрібно робити з урахуванням цього шуму.